Trabajo de investigación

Comparativa entre Listas Python y Arreglos Numpy

Agustin Samper
Departamento de Informática /Facultad de Ciencias Exactas Físicas y Naturales / Universidad Nacional de San Juan

Av. Ignacio de la Roza 590 (O), Complejo Universitario "Islas Malvinas", Rivadavia, San Juan,
Teléfonos: 4260353, 4260355 Sitio Web: http://www.exactas.unsj.edu.ar
e-mail: autor1@correo.com, autor2@correo.com

Resumen

El siguiente trabajo de investigación se encarga de realizar una comparación entre dos de las estructuras de datos más conocidas y populares que posee python, las cuales son las listas nativas de python y la librería de numpy. Se analizaron sus características principales, sus diferencias y sus ámbitos de uso. El trabajo incluye ejemplos prácticos haciendo uso de clases y objetos definidos por el programador. Este trabajo tiene como objetivo el brindar herramientas para seleccionar la estructura de datos adecuada en función de las necesidades del programador.

Palabras claves: Listas Python , Arreglos Numpy, Python, Estructuras de Datos, tipo de dato, Secuencias, Homogéneo, Heterogéneo.

Contexto

Esta investigación se realiza en el marco de la asignatura “Programación Orientada a Objetos” correspondiente al Departamento de Informática de la Facultad de Ciencias Exactas Físicas y Naturales de la Universidad Nacional de San Juan. El objetivo es profundizar en las estructuras de datos de python, brindando herramientas para distinguir sus diferencias y aplicaciones prácticas, utilizando documentación oficial y material académico brindado por la Universidad.

1. Introducción

Tanto las listas nativas de Python como los arreglos proporcionados por la librería Numpy constituyen estructuras de datos ampliamente utilizadas dentro de la comunidad de programación. Aunque comparten ciertas características, están orientadas a propósitos prácticos muy diferentes, debido a sus principales diferencias en el manejo de la información, la homogeneidad de los datos y su gestión en memoria.
A fin de identificar claramente sus ventajas y limitaciones, se presentan ejemplos prácticos basados en clases y objetos definidos por el programador. La investigación se apoya en documentación oficial y fuentes académicas reconocidas, con el objetivo de brindar una guía clara que facilite la correcta selección entre ambas estructuras según las necesidades específicas del problema.

2. Desarrollo

2.1 Listas Python

Las listas en Python son estructuras de datos basadas en secuencias, un concepto presente en el núcleo del lenguaje. Los tipos de datos basados en secuencias se caracterizan por ser una colección ordenada de elementos, que pueden ser homogéneos o heterogéneos.
En el caso particular de las listas, se destaca su heterogeneidad, es decir, la capacidad de almacenar elementos de diferentes tipos, como enteros, flotantes, cadenas de texto u objetos definidos por el programador.

Para gestionar sus elementos, las listas implementan el protocolo de secuencias de Python, que exige la definición de métodos especiales como ***__getitem__ y __len__***. El primero permite acceder a los elementos mediante índices (que comienzan en 0), mientras que el segundo devuelve la cantidad total de elementos contenidos en la lista.

Otro aspecto fundamental de las listas es su **mutabilidad**: los elementos pueden ser modificados, eliminados o añadidos en cualquier momento. Para estas operaciones, se utilizan métodos como ***.append(), .insert() y .extend()***, entre otros.

2.2 Ejemplo Práctico Listas

```
class Pais:
        __nombre: str
        __poblacion: int

        def __init__(self, nombre, poblacion):
        self.__nombre = nombre
        self.__poblacion = poblacion

        def getNombre(self):
        return self.__nombre

        def getPoblacion(self):
        return self.__poblacion

        def setNombre(self, nombre):
        self.__nombre = nombre

        def setPoblacion(self, poblacion):
        self.__poblacion = poblacion
```

```
from ClasePais import Pais

class ManejadorLista:
        __listaPaises: list

        def __init__(self):
        self.__listaPaises = []

        def test(self):
        # Creación de objetos
        pais1 = Pais("Argentina", 46000000)
        pais2 = Pais("Brasil", 213000000)
        pais3 = Pais("Chile", 19000000)

        # Uso de métodos básicos de manipulación
        self.__listaPaises.append(pais1)          # Agregar un elemento al final
        self.__listaPaises.append(pais2)
        self.__listaPaises.insert(1, pais3) # Insertar en la posición 1

        def mostrarElementos(self):
        # Recorrer la lista y mostrar los países
        for pais in self.__listaPaises:
        print("País: {}, Población: {}".format(pais.getNombre(), pais.getPoblacion()))

        # Eliminar un elemento de la lista
        self.__listaPaises.remove(pais2)

        print("\nDespués de eliminar:")
        for pais in self.__listaPaises:
        print("País: {}, Población: {}".format(pais.getNombre(), pais.getPoblacion()))
```

```
from ClaseManejadorLista import ManejadorLista

if __name__ == "__main__":
        manejador = ManejadorLista()
        manejador.test()
        manejador.mostrarElementos()
```

2.3 Arreglos Numpy

¿Qué son los Arreglos Numpy?

Los arreglos proporcionados por la biblioteca Numpy, al igual que las listas del núcleo de Python, son estructuras de datos ampliamente utilizadas. Surgieron con el propósito de realizar una fuerte optimización respecto a las listas nativas, ya que los arreglos Numpy son capaces de procesar enormes cantidades de datos de manera mucho más eficiente, sin presentar mayores inconvenientes.

Esta eficiencia se debe principalmente a dos factores:

- **Homogeneidad de los datos:** los arreglos de Numpy requieren que todos sus elementos sean estrictamente del mismo tipo. Esta restricción permite optimizar el almacenamiento y la velocidad de acceso. Aunque es posible almacenar objetos de una misma clase utilizando *dtype=object*, esta práctica restringe el uso de varios métodos matemáticos avanzados disponibles en Numpy, ya que están diseñados para operar principalmente sobre datos numéricos.

- **Implementación en C:** la biblioteca está escrita mayoritariamente en lenguaje C, lo que proporciona la ventaja de la compilación, permitiendo un procesamiento mucho más rápido y eficaz en operaciones de gran volumen.

Una de las mayores fortalezas de Numpy es la capacidad de realizar operaciones matemáticas **vectorizadas**, es decir, aplicar operaciones de manera simultánea sobre grandes bloques de datos, sin necesidad de realizar bucles explícitos en Python. Esto permite ejecutar cálculos complejos sobre millones de datos en tiempos muy reducidos en comparación a las listas comunes.

Entre los métodos más destacados y de uso frecuente en Numpy, se encuentran:

- ***.sum()****:* Permite obtener la suma de todos los elementos del arreglo, o bien de los elementos de un eje específico en arreglos multidimensionales.

- ***.reshape(shape)***: Reorganiza los datos en una nueva forma (dimensiones) sin modificar su contenido.

- ***.size():*** Devuelve la cantidad total de elementos contenidos en el arreglo.

- ***.mean()***: Calcula el valor medio de todos los elementos del arreglo.

- ***.max() y .min()***: Retornan respectivamente el valor máximo y mínimo presente en el arreglo.

- ***.astype(tipo)***: Permite convertir el tipo de datos del arreglo a uno nuevo especificado.

Estas operaciones son fundamentales para aprovechar al máximo las capacidades de Numpy y realizar procesamiento masivo de datos con un rendimiento notablemente superior al que ofrecen las listas tradicionales.

2.4 Ejemplo Práctico Numpy

```
import numpy as np
from ClasePais import Pais

class ManejadorArreglo:
        __arregloPaises: np.ndarray

        def __init__(self):
                self.__arregloPaises = np.empty(3, dtype=Pais)

        def test(self):
                # Creación de objetos
                pais1 = Pais("Argentina", 46000000)
                pais2 = Pais("Brasil", 213000000)
                pais3 = Pais("Chile", 19000000)

                # Asignar los países en el arreglo
                self.__arregloPaises[0] = pais1
                self.__arregloPaises[1] = pais2
                self.__arregloPaises[2] = pais3

        def mostrarElementos(self):
                # Recorrer el arreglo y mostrar los países
                for pais in self.__arregloPaises:
                        if pais is not None:
                                print("País: {}, Población: {}".format(pais.getNombre(),
        pais.getPoblacion()))

                # Eliminar un país (lo seteamos como None para simular eliminación)
                self.__arregloPaises[1] = None

                print("\nDespués de eliminar:")
                for pais in self.__arregloPaises:
                        if pais is not None:
                                print("País: {}, Población: {}".format(pais.getNombre(),
        pais.getPoblacion()))

from ClaseManejadorArreglo import ManejadorArreglo

if __name__ == "__main__":
        manejador = ManejadorArreglo()
        manejador.test()
        manejador.mostrarElementos()
```

```
País: Argentina, Población: 25000000
País: Francia, Población: 123000000
País: España, Población: 59000000

Después de eliminar:
País: Argentina, Población: 25000000
País: España, Población: 59000000
```

2.5 Cuadro Comparativo

Característica	***Listas de Python***	***Arreglos de Numpy***
Tipo de datos	*Heterogéneo (pueden almacenar distintos tipos)*	*Homogéneo (todos los elementos deben ser del mismo tipo)*
Flexibilidad	*Muy flexible (permite almacenar cualquier tipo de objeto)*	*Limitada (principalmente numéricos u objetos homogéneos)*
Velocidad	*Más lenta en operaciones masivas*	*Mucho más rápido para operaciones numéricas vectorizadas*
Consumo de memoria	*Mayor consumo de memoria debido a la flexibilidad*	*Más eficiente en memoria por el tipo de datos homogéneo*
Operaciones matemáticas	*No optimizadas, requieren bucles explícitos*	*Altamente optimizadas, operaciones vectorizadas*
Mutabilidad	*Mutable (se pueden modificar, agregar o quitar elementos)*	*Mutable (aunque menos flexible en forma directa)*
Facilidad de uso	*Muy intuitiva para principiantes*	*Puede requerir aprendizaje adicional*
Implementación interna	*Implementada puramente en Python*	*Implementada principalmente en C*
Soporte para multidimensionalidad	*Nativo sólo para listas de listas (más complejo de manejar)*	*Soporte nativo para matrices y arreglos multidimensionales*
Uso ideal	*Manejo de datos diversos y estructuras pequeñas*	*Procesamiento numérico intensivo y grandes volúmenes de datos*

3. Conclusiones

Las listas de Python y los arreglos de Numpy representan dos soluciones distintas para el almacenamiento y manipulación de datos, cada una adaptada a necesidades específicas.

Mientras que las listas ofrecen una gran flexibilidad al permitir la inclusión de distintos tipos de datos, su eficiencia disminuye notablemente cuando se requiere trabajar con grandes volúmenes de información o realizar operaciones numéricas intensivas.

Por otro lado, los arreglos de Numpy, gracias a su implementación en lenguaje C y su estructura optimizada, proporcionan un rendimiento superior en términos de velocidad de procesamiento y uso de memoria, aunque sacrifican cierta flexibilidad al exigir, por defecto, homogeneidad en los tipos de datos.

En función de estos aspectos, se concluye que la elección entre listas y arreglos debe basarse en el contexto y los objetivos del problema a resolver. Para tareas generales y heterogéneas, las listas son adecuadas; mientras que para operaciones matemáticas, científicas o de procesamiento masivo de datos, los arreglos de Numpy son ampliamente superiores.

4. Bibliografía

Python Software Foundation. (2024). *The Python Standard Library - Built-in Types: Lists*. Recuperado de https://docs.python.org/3/library/stdtypes.html#list

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., … Oliphant, T. E. (2020). *Array programming with NumPy*. *Nature*, 585(7825), 357–362. Recuperado de https://numpy.org/doc/

Ramírez, Ó. (2019). *Python a fondo*. México: Alfaomega Grupo Editor.